

Transferring a Dex3-1 manipulation policy to BrainCo Revo2 Touch hands

A GR00T N1.7 policy trained on a Unitree G1 with 7-DoF Dex3-1 hands, made to drive the same robot wearing 6-DoF BrainCo Revo2 Touch hands — first by retargeting alone, then by fine-tuning on demonstrations the retargeted policy generated for itself. Each step folder has its own README with the commands; this is what was done and what came out of it.

Analytic retargeting takes the task from 0.00 to **0.06**, against a source-embodiment baseline of **0.65**, and the ablation shows no hand-side change alone produced a single success. That residual gap is behavioural rather than geometric, so fine-tuning on 1,668 self-generated demonstrations recovers it to **0.50 under randomised spawns** — an eight-fold gain, statistically indistinguishable from the Dex3 baseline despite harder scoring. Where that policy then stops working is the more useful half of the result.

Platform, task, and what the two hands differ by

The requirement was a pretrained policy whose hands could be swapped for a target with a different DoF count. Three candidates failed on inspection, leaving IsaacLab-Arena's `galileo_g1_static_pick_and_place` correct robot, 7 DoF per hand against the Revo2's 6, no locomotion, a published checkpoint reproducing at 0.65. Two limits matter later — the task is single-handed in practice (the right-hand action is identically zero), and **the apple spawn is deterministic**, so everything the fine-tuned policy knows about **where** the apple is comes from jitter added deliberately during data generation.

The policy runs at 50 Hz on a 480×640 head camera, a 43-D joint state and a 50-D action, feeding an AGILE whole-body controller. None of that changes between runs; the policy is not aware the hands changed. The Dex3-1 has three fingers and seven actuators; the Revo2 has five fingers and six, with five distal joints coupled through URDF `<mimic>` tags. Beyond the count, measured by forward kinematics on both URDFs rather than from datasheets: fingertips at equal closure differ by 2–3 cm; ring and pinky have no source signal and `thumb_0` no target counterpart; closing takes ~0.65 s against ~0.2 s; hand mass drops 1.39→0.29 kg; maximum aperture is 193.3 against 140.6 mm and the grasp centres sit 25.2 mm apart.

Angle conventions are irreconcilable — Dex3 joints are sign-mirrored with negative "closed" limits, every Revo2 joint runs 0 to positive — so the map is defined on **closure fraction**: index, middle and thumb collapse to one grasp closure, and ring and pinky follow their mean, the one place the map invents information. Four things then have to be right at once, each found through a failure:

- **Aperture calibration.** Matching fractions leaves a 33 mm gap where the Dex3 pinches to 11.7 mm, so the hand closes on empty air. Openings are matched on the **running minimum**, since both aperture curves turn back up once the fingers curl past the thumb.
- **Thumb opposition.** The Revo2's metacarpal sweeps across the palm rather than rolling, so a searched schedule puts it into opposition ahead of the fingers: pinch aperture at 75% closure goes 36.9 → 5.5 mm, and without it no grasp is geometrically possible.
- **Invertibility.** Strict matching wants zero Revo2 closure below Dex3 closure ≈ 0.35 , a closed-loop hazard — the policy commands a closure, nothing moves, its next observation reports an open hand. Lifting those entries took round-trip error 0.300 → 0.000.
- **Interface preservation.** The action space stays 50-D in Dex3 coordinates and the 41-DoF articulation is reached only inside the action term, so the policy, its client, the modality config and the controller are untouched — which is what later makes fine-tuning a drop-in.

The wrist adapter belongs here too: the policy aims the wrist having learned where a Dex3 palm sits relative to it, and at the stock mount the Revo2's grasp centre is 25.2 mm away, so the hand closes above the apple's equator and rolls it. Applied in the URDF; on hardware a machined plate.

Isaac Lab's URDF converter proved unfaithful in three ways, all fixed and all raising errors: a welded pelvis, instanceable links that stop the task's friction material binding to the fingers, and absent joint-velocity caps. A fourth raised nothing at all — the converter nests links under a Geometry scope, so the default camera path matches no prim and USD quietly creates an empty Xform at the robot origin. **The head camera sat at the pelvis staring at the robot's own arms — and the policy is a vision model.** Fixed, then verified against the baseline stream.

Results: retargeting alone

	Dex3-1 baseline	Revo2, retargeting only
Task success rate	0.65 (13/20)	0.06 (6/100)
Frame-verified pick-and-place	not audited	0.06, all six genuine
Object moved rate	0.65	0.21

Identical policy, task, scene and evaluation script; the only difference is --embodiment. The task counts the apple being **pushed** into the plate region, and in one run only 1 of 3 counted successes was a real grasp, so every number here was checked frame by frame. Object-moved separates "never reaches the apple" from "reaches but cannot hold it", and the first three configurations touch the apple in zero episodes. **No gripper-side change alone produced a single success**; the two that moved the needle were the wrist adapter and the pelvis collider — mount geometry and physics.

The pelvis was the most instructive failure. It crept 5.7 cm backwards every episode and settled 2.4 cm lower, turning the top-down grasp into a side sweep that pushes the apple. Everything authored matched, so I compared the runtime values PhysX actually loaded: an invisible collision slab sits under the table, the robot spawns overlapping it, and the stock G1 braces against it with a `pelvis_contour_link` collider the supplied BrainCo URDF omits. What looked like drift was the free-standing controller doing its job. Restoring the link took true pick-and-place 1/100 → 6/100, after four more plausible explanations were rejected.

What remained was grasp slip: the hand reaches the apple and closes, but the contact patch is a five-finger cage against an opposed thumb rather than a three-finger pinch, and the apple rolls out during the lift. A better analytic map can align where fingertips are; it cannot change that five fingers cage a sphere where three pinch it, nor make a 0.65 s actuator arrive at a grasp timed for a 0.2 s one. Both are closed-loop behaviour, and both are what demonstrations show directly — the argument for fine-tuning, made on the ablation rather than on a hunch.

Demonstrations without a teleoperator, and what fine-tuning bought

Successes are harvested straight from the retargeted policy — Arena's recorder stamps the success term and discards failures, so a rollout loop yields success-filtered demos for free. That returned 36 successes from 553 episodes (6.5%), independently confirming the measured 0.06, and those became 22 Isaac Lab Mimic source demos. Mimic regenerates each against randomised scenes, with per-episode spawn jitter (the stock environment spawns at a fixed pose, so without it Mimic would add only action noise) and grasp-depth resampling; running it meant rebuilding the source demos as 23-D Pink IK actions and writing the subtask graph the static task ships without. One property matters later: **the harvester filters on success**, so nothing that failed and recovered can be in the data.

Both fine-tunes start from NVIDIA's task-tuned GN1x-Tuned-Arena-G1-Static-PickNPlace— the same weights evaluated frozen above — and follow Arena's recipe: adapt the vision tower, projector and flow-matching action head, leave the language model frozen.

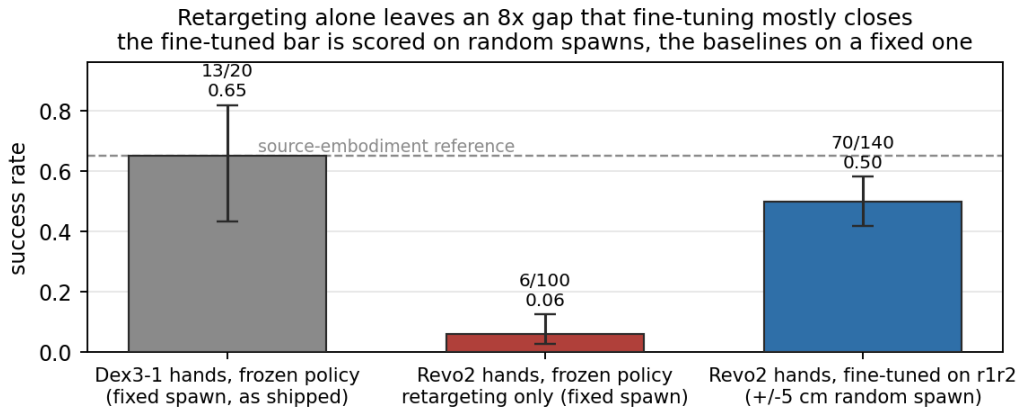


Figure 1. The fine-tuned bar is the randomised-spawn run. Scoring it at the fixed spawn would be scoring it on its own training point.

Fine-tuning on 1,668 episodes (5,000 steps, batch 192) took the task from **0.06 to 0.50** (70/140), $p = 5 \times 10^{-13}$. That 0.50 is measured with **± 5 cm of random spawn jitter** while both frozen baselines were scored at the single fixed pose the task ships with, so the comparison runs against the fine-tune and the gain is a lower bound. Against the 0.65 Dex3 baseline it is not separable at this sample size ($p = 0.21$): retargeting plus self-generated data brings a 6-DoF hand to within noise of the 7-DoF hand the policy was trained on, under strictly harder evaluation. I deliberately do **not** report the fine-tuned score at the fixed spawn — that pose is the one all 1,668 demonstrations were generated around, so asking for it again tests the policy on its own training point and measures memorisation of one apple position. It is kept in results.json flagged reportable: false.

Where it fails: apple spawn position versus success

Success is 0.50 at ± 5 cm and **0.00 at ± 10 cm**: the task was learned, the workspace was not. To find out **where**, I recovered each episode's apple spawn from the recorded video through a pixel-to-world homography (± 1 cm, so boundaries are approximate) and plotted it against the outcome.

The fine-tuned policy's failures are positional, not random

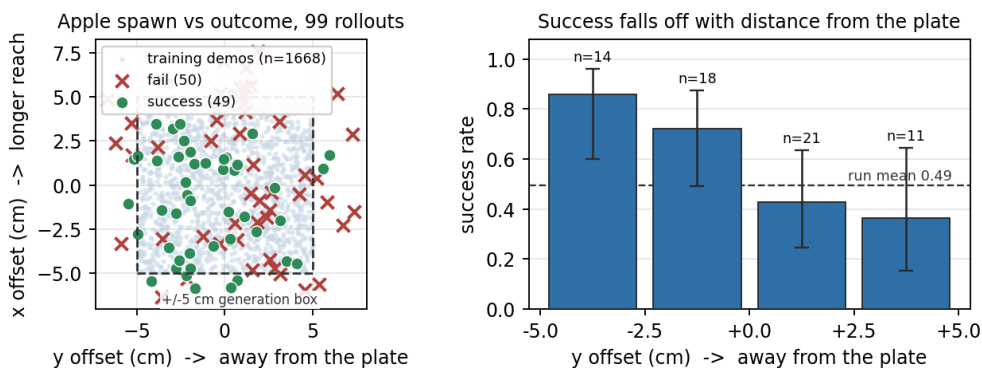


Figure 2. Every evaluation episode by spawn position and outcome, over the cloud of training spawns; and the same episodes collapsed onto the axis that carries the structure.

The failures are positional, not scattered. Inside the generation box success runs **25/32 = 0.78 in the half nearer the plate against 13/32 = 0.41 in the far half** ($p = 0.0023$), decaying monotonically with distance from the plate, $0.86 \rightarrow 0.72 \rightarrow 0.43 \rightarrow 0.36$. Outside the box, where no demonstration was ever generated, it is close to hopeless — the ± 10 cm collapse, seen episode by episode.

That gave me a map to aim at. I binned those 99 episodes onto a 3×3 grid, binned all 1,668 demonstrations onto the same grid, and — because every rollout is a real-time simulation — replayed the checkpoint offline against 400 held-out episodes at their exact spawns, hoping its own action error would reproduce the success map cheaply.

Success varies 6x across the box, while the model's own offline error varies 1.4x and demo coverage 1.2x

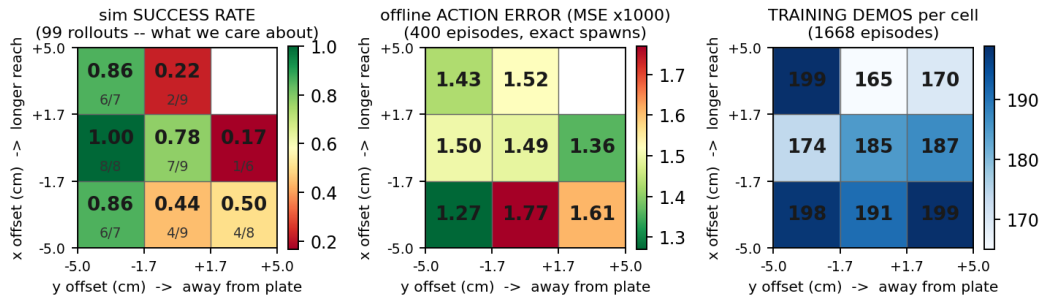


Figure 3. Three quantities over the same spawn box. Only the first has any spatial structure.

It does not. Success varies **six-fold** (1.00 near the plate at moderate reach to 0.17 at the far edge) while offline action error spans 1.4x with no spatial structure — correlation -0.06 with distance from centre, $+0.03$ with distance from the plate, and -0.24 against actual success over eight cells, which at that n is noise. **The quantity the model is trained to minimise is blind to where it fails**, and the same holds along the training axis: offline MSE falls monotonically $2.14 \rightarrow 1.49 \times 10^{-3}$ between steps 2000 and 5000 while success bounces 0.37–0.58 with no trend. Everything here is therefore scored by rollout.

So I generated **round 3** against the success map — 979 demonstrations in proportion to per-cell failure, using a Beta(1,1) posterior mean so a cell that went 0/3 could not swallow the budget, and zero budget above 0.70. Merged to 2,647 episodes, 10,000 steps at batch 240. **It bought nothing measurable**: 0.55 at step 8000 and 0.50 at step 10000 against the 0.50 r1r2 had already reached, a two-proportion p of 1.0, flat across all six evaluated checkpoints of both runs. The third panel is why, and it is what I should have read off it first. **Coverage was already uniform** — every cell held 165–199 demonstrations, a 1.2x spread, while success over those same cells ranged 0.17 to 1.00. The weak cells were equally represented and still failed, so round 3 treated a kinematic problem as a data-density problem: what varies across that box is reach and wrist orientation, and more replays of the same 22 trajectories add scene diversity, not approach diversity. Two caveats — r2r3 was **never evaluated at ± 10 cm**, and 1.8 epochs is not much training for a spatial skill. The loss cannot arbitrate either: r2r3 reaches a *lower* loss (0.0103 against 0.0139) while being no better.

Why the policy never retries

Every remaining failure has one shape. The policy reaches the apple, closes the hand, the apple rolls out during the lift — and it carries on to the plate and mimes a place with an empty hand until the clock runs out. On the final checkpoint all six failures ran the full 14 s against successes finishing in 5.1–8.5 s. It never notices and never tries again.

That pointed at inference: the policy predicts 40 actions (0.8 s) and the stock loop executes all 40 before looking at the world again, so a grasp failing at frame 5 is not seen until frame 40. Three variants on the same checkpoint — a shortened chunk with hard swap at 0.2 s ($16/40 = 0.40$, $p = 0.26$), ACT-style temporal ensembling at 0.2 s ($25/40 = 0.62$, $p = 0.50$), and real-time chunking at 0.1/0.2/0.4 s (10/16, 6/13, 9/15) — **produced not one retry between them**. Why matters more than the numbers: ensembling *averages* overlapping plans, and the average of "continue the lift" and "re-grasp" is neither; RTC inpaints each chunk from the previous one's un-executed tail and ramps velocity across the overlap **specifically** so consecutive plans agree — it is built to suppress abrupt plan changes, and a retry is exactly that. I built two mechanisms that make recovery harder.

So I asked whether retry behaviour was in the data at all. The grasp command is binary — dimension 23 is exactly 0.0 or -0.87 rad — so counting rising edges is unambiguous. **All 2,647 episodes close the hand exactly once. Zero re-grasps**. Structural rather than a sampling artefact: the harvester filters on success, so an episode containing a fumble-and-recover could never have entered the dataset. One limit, since NVIDIA's card says the base checkpoint saw 200 human-teleoperated demonstrations, which often do contain fumbles: I cannot assert the base model never held a recovery prior. The measured claim is narrower and sufficient — our data contains no recovery, and our fine-tune trained the action head hard on our data.

Conclusion

Retargeting alone does not get this policy back to baseline. Everything a frozen policy allows took the task $0.00 \rightarrow 0.06$, with no hand-side change alone producing a single success. That localised the gap as behavioural, and fine-tuning confirmed it: **0.06 $\rightarrow$ 0.50 under randomised spawns**, within noise of the 0.65 source-embodiment baseline, with no teleoperation and no human demonstrator anywhere in the pipeline. The policy's own 6% was the seed and Mimic the multiplier.

The more valuable half is the three things that did not work, each measured. **A targeted second dataset round did not help** ($0.50 \rightarrow 0.55$, $p = 1.0$), and the spawn analysis shows it could not have. **The loss stopped being informative after ~3,000 steps. No inference-time strategy produced recovery**, two of the three being structurally incapable of it — tying back to 2,647 episodes, 2,647 single grasps, zero retries. The policy behaves as a faithful clone of its data should, and the ceiling belongs to the data-generation pipeline rather than to the model, recipe or inference loop.

What I would do next is **diversify the grasp itself**. Every number above rests on 22 source trajectories replayed into different scenes, so the data has scene diversity and almost no approach diversity — one wrist orientation, one approach direction, one closure depth — which is exactly the axis along which success varies six-fold across the spawn box. Harvesting many more distinct source trajectories, widening the generation box and sampling wrist orientation and approach direction per episode attacks the measured failure where another 979 replays did not. Alongside it, **generate demonstrations that contain recovery**, by not filtering on success or by scripting a fumble into the Mimic sources, since no inference-time trick can surface a behaviour absent from the data.

One option I considered and rejected: **adding cameras to the hands**. Wrist or palm views would plausibly help, since the failure is a slip the policy cannot see. But the assignment is to swap the hands with minimal change to the policy, and a new camera changes the observation space — a new encoder input with no pretrained weights, and a baseline that no longer shares an interface with the Dex3 policy, so the retargeting comparison loses its reference. The same applies to the Revo2's tactile sensing, unused here and exactly the signal that would catch the slip: consuming it is a policy-architecture change, not a hand swap. That is this work's boundary, not an oversight.